

COMICCRAFT – AI

Comic Story Creator Using Gemini Models

SET UP THE DEVELOPMENT ENVIRONMENT

Detailed 20-Page Project Report

Project Domain	Generative AI / Creative Application
Core Model	Gemini-family generative AI models
Backend	Python + FastAPI
Frontend	Web-based comic creation interface
Environment	Python virtual environment + Git
Primary Focus	Development environment setup

Prepared as a comprehensive academic/project-development document covering brainstorming, requirements, design, planning, development, testing, documentation, and demonstration.

1. Executive Overview and Purpose

ComicCraft is an AI-assisted application intended to transform a user's story idea into a structured comic experience. The system can accept a concept, characters, setting, dialogue requirements, tone, and visual directions, then coordinate generative AI capabilities to produce comic-oriented content. The purpose of this report is specifically to describe how the development environment should be established before core implementation begins. A reliable environment is essential because an AI application depends on several moving parts: Python packages, API credentials, backend routing, model access, frontend assets, prompt templates, file handling, logging, testing utilities, and version control.

Detailed implementation and verification points:

- Define a reproducible workstation setup for all team members.
- Create an isolated Python environment so project dependencies do not conflict with other applications.
- Install and verify the framework libraries required by the ComicCraft backend.
- Establish secure configuration management for API keys and environment-specific settings.
- Prepare a clean project directory structure that supports future growth.
- Prepare Git-based version control before significant code is written.
- Create development, testing, and demonstration workflows that can be repeated reliably.
- Document every environment assumption so another developer can reproduce the setup.
- Keep generated content, temporary files, logs, and secrets separated from source code.
- Provide a foundation for later Gemini model integration without hard-coding provider credentials.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

2. Brainstorming and Ideation

The first phase converts a broad creative idea into a practical engineering problem. ComicCraft can be imagined as a pipeline: the user supplies a story concept, the system validates the input, a generative model expands the concept into structured scenes, and the application presents the result as a comic plan. During brainstorming, the team should avoid immediately committing to implementation details. Instead, it should identify the desired user experience, the minimum useful feature set, and the technical constraints that affect environment setup.

Detailed implementation and verification points:

- Identify the target users: students, creators, educators, hobbyists, or prototype demonstrators.
- List the main user action: enter a story idea and generate a comic-ready story.
- Identify optional controls such as genre, number of panels, characters, tone, and language.
- Consider whether output should be plain text, structured JSON, downloadable files, or images.
- Discuss how AI failures should be shown to users without exposing internal stack traces.
- Define what information must remain private, especially API keys and user-provided content.
- Identify likely external dependencies such as a Gemini API client and web framework.
- Decide which features belong in the first demonstrable version.
- Record alternative ideas without allowing optional features to block the core prototype.
- Convert brainstorming results into measurable requirements and acceptance criteria.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

3. Requirement Analysis

Requirement analysis converts ideas into testable statements. ComicCraft requires both functional and non-functional requirements. Functional requirements describe what the application does, while non-functional requirements describe qualities such as reliability, maintainability, security, performance, and reproducibility. The development environment must satisfy these requirements before application coding begins.

Detailed implementation and verification points:

- FR-01: The application shall accept a user story prompt.
- FR-02: The backend shall validate required input fields.
- FR-03: The application shall call the configured generative AI service through a controlled integration layer.
- FR-04: The response shall be converted into a predictable comic-story structure.
- FR-05: The system shall provide useful error messages for invalid requests and provider failures.
- FR-06: The application shall expose a development endpoint for health checking.
- NFR-01: Dependencies shall be isolated inside a project virtual environment.
- NFR-02: Secrets shall be supplied through environment configuration rather than committed source code.
- NFR-03: Source code shall be version-controlled.
- NFR-04: The project shall include reproducible setup instructions.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

4. Project Design

The proposed design separates presentation, API routing, business logic, model integration, configuration, and utility functions. This separation makes the environment easier to test because individual components can be replaced or mocked. A practical architecture contains a web interface, a FastAPI service, an application/service layer, a Gemini integration module, configuration management, and a structured output layer. The design should avoid placing model calls directly inside route functions.

Detailed implementation and verification points:

- Presentation layer: collects story parameters and displays generated comic content.
- API layer: receives HTTP requests and returns validated responses.
- Service layer: manages story generation workflow and prompt construction.
- Model layer: communicates with the configured Gemini API client.
- Schema layer: validates request and response structures.
- Configuration layer: reads environment variables and application settings.
- Utility layer: handles logging, file paths, identifiers, and common helpers.
- Testing layer: provides unit and integration tests around each major boundary.
- Documentation layer: explains setup, API usage, architecture, and demonstration steps.
- Repository layer: stores source code while excluding secrets, caches, and generated artifacts.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

5. Project Planning

A development environment should be prepared according to a sequence rather than through random package installation. Planning reduces setup errors and makes troubleshooting easier. The recommended order is requirements discovery, tool installation, repository creation, Python environment creation, dependency installation, configuration setup, skeleton application creation, verification, testing, and documentation. Each stage should have a simple completion condition.

Detailed implementation and verification points:

- Milestone 1: Confirm operating-system prerequisites and supported Python version.
- Milestone 2: Install a code editor or IDE and Git.
- Milestone 3: Create the repository and initial README.
- Milestone 4: Create and activate the Python virtual environment.
- Milestone 5: Install FastAPI, an ASGI server, configuration utilities, and testing tools.
- Milestone 6: Add the Gemini SDK/client required by the selected integration approach.
- Milestone 7: Create environment-variable templates without exposing real credentials.
- Milestone 8: Build a minimal health-check endpoint.
- Milestone 9: Run local tests and verify the development server.
- Milestone 10: Record the final setup procedure for demonstration and handover.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

6. Development Environment Prerequisites

The workstation should have a modern operating system, a supported Python installation, Git, a code editor, and reliable internet access. The exact Python and SDK versions should be pinned or recorded after validation because package compatibility can change over time. The environment should be created per project rather than relying on globally installed Python packages. This approach minimizes dependency collisions and makes the project easier to reproduce.

Detailed implementation and verification points:

- Install Python from a trusted official distribution or managed package source.
- Verify the Python interpreter with a version command before creating the project.
- Install Git and verify that the Git executable is available from the terminal.
- Choose an editor with Python support, syntax highlighting, and integrated terminal access.
- Use a dedicated project folder with a simple path and no unnecessary permissions.
- Ensure the workstation can reach the required package repositories.
- Ensure the application can reach the selected Gemini service when integration testing is performed.
- Avoid storing API keys directly in source files or screenshots.
- Create a separate location for generated output if large files are expected.
- Record operating-system and tool versions in the project documentation.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

7. Installing and Verifying Python

Python is the primary development language in this architecture. A clean installation should be followed by verification of the interpreter, package manager, and virtual-environment support. The project should use one clearly identified Python executable so commands are not accidentally executed against a different installation. If multiple Python versions exist, the team should explicitly select the version used by the project.

Detailed implementation and verification points:

- Open a terminal and check the installed Python version.
- Confirm that the Python package manager is available.
- Confirm that the built-in virtual-environment module can create an environment.
- Create the ComicCraft project directory.
- Create a virtual environment named `.venv`.
- Activate `.venv` before installing project packages.
- Upgrade the package installer inside the virtual environment.
- Run a small Python command to verify that the interpreter responds correctly.
- Record the chosen Python version in `README.md`.
- Do not depend on packages installed outside the virtual environment.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Example verification commands

```
python --version<br/>python -m pip --version<br/>python -m venv .venv
```

8. Creating the Project Repository

Version control should begin before the application becomes large. A repository provides a history of changes and a controlled place for source code, documentation, tests, and configuration templates. The initial repository should be intentionally small. A clear structure makes it easier for students, reviewers, and future contributors to understand where each responsibility belongs.

Detailed implementation and verification points:

- Create a Git repository for ComicCraft.
- Create a README.md with project purpose and basic setup.
- Create a .gitignore file before adding local environment files.
- Ignore .venv, Python cache directories, local secrets, logs, and generated temporary files.
- Create an app or src directory for application code.
- Create a tests directory for automated tests.
- Create a docs directory for project documentation.
- Create a .env.example template containing variable names but no secret values.
- Make an initial commit containing only safe project scaffolding.
- Use meaningful commit messages as the project evolves.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Example Git initialization

```
git init<br/>git status<br/>git add .<br/>git commit -m "Initialize ComicCraft development environment"
```

9. Virtual Environment and Dependency Management

A virtual environment provides project-level isolation. This is particularly important for AI applications because SDKs, web frameworks, validation libraries, and testing tools may evolve independently. Dependency management should distinguish between direct dependencies and development-only tools. The final project should include a reproducible dependency file or equivalent lock strategy suitable for the chosen workflow.

Detailed implementation and verification points:

- Create `.venv` inside the project directory or use an agreed external environment location.
- Activate the environment before installing dependencies.
- Install only packages actually required by the project.
- Separate runtime packages from testing and development packages when practical.
- Freeze or lock validated dependency versions for reproducibility.
- Recreate the environment from the dependency specification as a verification exercise.
- Remove unused packages instead of keeping them as unexplained dependencies.
- Document activation commands for Windows and Unix-like systems.
- Never commit the `.venv` directory to Git.
- Test the setup on a second machine or clean environment before final demonstration.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Example environment activation (Windows)

```
.venv\Scripts\activate
```

Example environment activation (macOS/Linux)

```
source .venv/bin/activate
```

10. Installing FastAPI and Server Components

FastAPI provides the HTTP API layer for ComicCraft. An ASGI server runs the application locally during development. The environment should verify that the framework imports successfully and that a minimal route can be served. Starting with a health endpoint is useful because it proves the basic Python, framework, routing, and server layers work before an AI provider is introduced.

Detailed implementation and verification points:

- Install FastAPI and the selected ASGI server inside the virtual environment.
- Create a minimal application object.
- Add a GET health endpoint that returns a small JSON response.
- Start the development server using the documented command.
- Open the local API endpoint in a browser or API client.
- Check the automatically generated API documentation.
- Stop and restart the server to verify the command is reproducible.
- Keep development reload enabled only where appropriate for local work.
- Avoid putting long-running model calls directly into unrelated startup code.
- Record the server command and expected response in the README.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Example local server command

```
uvicorn app.main:app --reload
```

11. Gemini Integration Preparation

ComicCraft's generative AI capability requires an integration boundary between the application and the model provider. The environment should therefore prepare configuration and SDK access without embedding credentials. The exact SDK package, model identifier, authentication method, quotas, and available capabilities should be checked against the current provider documentation when implementation begins. A provider abstraction helps keep the rest of the application independent from one specific client library.

Detailed implementation and verification points:

- Create a configuration variable for the model-provider API credential.
- Create a configuration variable for the selected model identifier.
- Load configuration through environment variables or a secure configuration mechanism.
- Keep .env files containing real secrets out of version control.
- Provide .env.example with safe placeholder values.
- Create a model service module rather than calling the provider from every route.
- Add timeouts and controlled error handling around external requests.
- Validate the response before passing it to the rest of the application.
- Log operational information without logging secrets or sensitive prompt content unnecessarily.
- Document where developers obtain credentials according to the provider's official process.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Example safe configuration template

```
GEMINI_API_KEY=replace_with_local_secret<br/>GEMINI_MODEL=replace_with_valid_model_id<br/>APP_ENV=development
```

12. Configuration, Secrets, and Environment Variables

Configuration should be treated as data, not hard-coded application logic. Local development may use a .env file, while deployment environments may inject variables through a secret manager or platform configuration. The same source code should work across environments by changing configuration values rather than editing Python files. This also makes demonstrations safer because credentials can be rotated without changing application logic.

Detailed implementation and verification points:

- Define APP_ENV to distinguish development, testing, and production-like contexts.
- Define the model-provider credential variable.
- Define the model identifier variable.
- Define optional logging level and application host/port variables.
- Validate required variables when the application starts or when the relevant service is used.
- Provide safe default values only for non-secret settings.
- Fail clearly when a required secret is missing.
- Never print the complete API key to logs.
- Use .gitignore to exclude local secret files.
- Rotate exposed credentials immediately if they are accidentally committed or shared.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

13. Recommended Folder Structure

A clear folder structure reduces the cognitive load of development. The exact naming can vary, but responsibilities should remain distinct. A practical structure is shown below. It is intentionally simple enough for an academic project while leaving room for growth into a production-oriented architecture.

Detailed implementation and verification points:

- comiccraft/
- ■■■■ app/ — application package and backend logic
- ■■■■ app/api/ — API route modules
- ■■■■ app/core/ — configuration and common infrastructure
- ■■■■ app/models/ — request/response schemas and domain models
- ■■■■ app/services/ — story generation and model-provider services
- ■■■■ tests/ — unit and integration tests
- ■■■■ docs/ — architecture, setup, and demonstration notes
- ■■■■ .env.example — safe configuration template
- ■■■■ .gitignore — ignored local files
- ■■■■ requirements.txt or pyproject.toml — dependency specification
- ■■■■ README.md — project setup and usage instructions

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Reference tree

```
comiccraft/<br/>&nbsp;&nbsp;&nbsp;app/<br/>&nbsp;&nbsp;&nbsp;tests/<br/>&nbsp;&nbsp;&nbsp;docs/<br/>&nbsp;&nbsp;&nbsp;.env.example<br/>&nbsp;&nbsp;&nbsp;.gitignore<br/>&nbsp;&nbsp;&nbsp;README.md
```

14. Core Application Skeleton

Once the environment is ready, the team can build a small application skeleton. The first objective is not to generate a full comic. It is to prove that the project can start, accept a request, validate it, and return a controlled response. A minimal skeleton makes later failures easier to diagnose because each layer can be verified independently.

Detailed implementation and verification points:

- Create the FastAPI application object.
- Create a health route for environment verification.
- Create a story-generation request schema.
- Create a story-generation response schema.
- Create a service interface for generative model calls.
- Create a provider implementation behind that interface.
- Add centralized exception handling for expected application errors.
- Add structured logging for development diagnostics.
- Keep route handlers thin and delegate business logic to services.
- Write a first unit test for the health endpoint and a validation test for the request schema.

Additional project-quality considerations:

- **Verification checkpoint:** confirm the step works before moving to the next stage.
- **Traceability:** connect the environment decision to a requirement, test, or documented project outcome.
- **Reproducibility:** record the command, dependency, version, or configuration assumption used.
- **Maintainability:** prefer small modules and clear interfaces over one large application file.
- **Security:** keep credentials outside source control and avoid unnecessary exposure in logs.
- **Reliability:** isolate external-provider failures from local validation and routing failures.
- **Testing principle:** use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- **Documentation principle:** update setup instructions whenever the actual development procedure changes.
- **Team principle:** use consistent naming, formatting, repository conventions, and commit practices.
- **Academic principle:** preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- **Operational principle:** a successful local health check should be completed before diagnosing model-provider problems.
- **Quality principle:** every environment dependency should have a clear reason for being present.

Minimal FastAPI skeleton concept

```
from fastapi import FastAPI
app = FastAPI(title="ComicCraft")
@app.get("/health")
def health():
    return {"status": "ok"}
```

15. Development Workflow

A disciplined development workflow helps prevent environment drift. Developers should work in small changes, run tests frequently, and keep documentation synchronized with the actual commands. AI projects benefit particularly from this discipline because prompt changes, model changes, and dependency changes can affect behavior independently.

Detailed implementation and verification points:

- Pull or clone the current repository before beginning work.
- Activate the correct virtual environment.
- Install or synchronize dependencies from the project specification.
- Create a focused branch or task change when using collaborative Git workflows.
- Implement one small feature at a time.
- Run formatting, linting, and tests where configured.
- Start the local server and manually verify affected endpoints.
- Review logs for unexpected warnings or leaked configuration values.
- Update documentation when setup or behavior changes.
- Commit a coherent change with a descriptive message.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

16. Testing the Development Environment

Environment testing proves that the setup itself works, not merely that the source code exists. Tests should begin with import checks and health endpoints, then expand to request validation, mocked model responses, provider error handling, and integration tests where credentials are available. External AI calls should not be required for every unit test because that would make the suite slow, costly, and dependent on network availability.

Detailed implementation and verification points:

- Test that the application imports without errors.
- Test that the health endpoint returns the expected status.
- Test valid story-generation input.
- Test missing and invalid fields.
- Mock the model service for deterministic unit tests.
- Test handling of provider timeouts and malformed responses.
- Run integration tests separately when real API credentials are available.
- Verify that secrets do not appear in test output.
- Run the test suite in a clean virtual environment.
- Record the commands and expected outcomes in the documentation.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Example test command

```
pytest -q
```

17. Troubleshooting and Risk Management

Development environments fail for predictable reasons: wrong interpreter, inactive virtual environment, incompatible packages, missing credentials, network restrictions, incorrect environment variables, or provider-side limits. A troubleshooting plan should diagnose the simplest local causes first. The objective is to make failures informative rather than mysterious.

Detailed implementation and verification points:

- If an import fails, verify that the correct virtual environment is active.
- If a command is not found, verify installation and PATH configuration.
- If a package conflicts, recreate the virtual environment from the dependency specification.
- If authentication fails, verify the environment variable name and credential status without printing the secret.
- If the model request fails, inspect the HTTP/provider error category and configured model identifier.
- If the server does not start, check port usage and syntax errors.
- If tests behave differently across machines, compare Python and dependency versions.
- If output is inconsistent, separate environment problems from normal generative-model variability.
- Maintain a troubleshooting section in README.md.
- Use version control so a known-good setup can be restored.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

18. Documentation Strategy

Documentation is part of the development environment, not an afterthought. A good project report should allow a new developer to move from an empty workstation to a running local application. Documentation should also distinguish commands that are universal from commands that depend on operating system, shell, package manager, or provider configuration.

Detailed implementation and verification points:

- README: project overview, prerequisites, installation, configuration, running, and testing.
- SETUP GUIDE: step-by-step environment creation with verification checkpoints.
- ARCHITECTURE: component responsibilities and request flow.
- API GUIDE: endpoint descriptions, request fields, response examples, and errors.
- PROMPT GUIDE: prompt templates, structured output expectations, and revision notes.
- TESTING GUIDE: unit, integration, and manual verification procedures.
- DEMO GUIDE: exact steps for presenting the working application.
- TROUBLESHOOTING: common failures and diagnostic commands.
- CHANGELOG: significant changes to dependencies and behavior.
- SECURITY NOTES: credential handling, logging, and secret-rotation procedure.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

19. Demonstration Preparation

A strong demonstration should prove the environment before showcasing the AI feature. The presenter should begin with the repository and environment structure, then show the application starting locally, the health endpoint, the API documentation, and finally a sample story-generation workflow. A prepared demonstration dataset reduces the risk of spending the presentation time typing long prompts or debugging accidental input errors.

Detailed implementation and verification points:

- Open the project repository and show the documented structure.
- Show the Python virtual environment and dependency specification.
- Start the FastAPI development server.
- Open the API documentation generated by FastAPI.
- Call the health endpoint and show a successful response.
- Provide a short, safe example story prompt.
- Demonstrate validation using an intentionally incomplete request if useful.
- Demonstrate a successful model-backed request when credentials and network access are available.
- Explain that generated output can vary because it is produced by a generative model.
- Conclude by showing the README setup steps and test results.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

20. Final Checklist, Outcomes, and Future Expansion

The development environment is complete when another developer can obtain the repository, install the documented prerequisites, create the virtual environment, install dependencies, configure credentials safely, start the server, run the tests, and reproduce the documented demonstration. This final checklist connects all eight requested project activities and establishes a practical baseline for future ComicCraft development.

Detailed implementation and verification points:

- Brainstorming and ideation produced a clear application concept and first-release scope.
- Requirement analysis converted the concept into functional and non-functional requirements.
- Project design separated the API, services, model integration, schemas, configuration, and utilities.
- Project planning established milestones and verification checkpoints.
- Development preparation established Python, Git, FastAPI, testing, and configuration foundations.
- Testing strategy covers deterministic unit tests plus optional real-provider integration tests.
- Documentation strategy captures setup, architecture, API behavior, testing, security, and demonstration.
- Demonstration preparation provides a repeatable sequence for presenting the project.
- Future work can add persistent projects, character libraries, panel layout generation, image generation, export formats, authentication, and analytics.
- Future work should preserve the same principles: isolated dependencies, secure configuration, testable components, clear documentation, and reproducible setup.

Additional project-quality considerations:

- Verification checkpoint: confirm the step works before moving to the next stage.
- Traceability: connect the environment decision to a requirement, test, or documented project outcome.
- Reproducibility: record the command, dependency, version, or configuration assumption used.
- Maintainability: prefer small modules and clear interfaces over one large application file.
- Security: keep credentials outside source control and avoid unnecessary exposure in logs.
- Reliability: isolate external-provider failures from local validation and routing failures.
- Testing principle: use deterministic mocks for routine tests and reserve live calls for controlled integration checks.
- Documentation principle: update setup instructions whenever the actual development procedure changes.
- Team principle: use consistent naming, formatting, repository conventions, and commit practices.
- Academic principle: preserve screenshots, test evidence, diagrams, and version information for the final project demonstration.
- Operational principle: a successful local health check should be completed before diagnosing model-provider problems.
- Quality principle: every environment dependency should have a clear reason for being present.

Release-readiness checklist

- Clean clone succeeds.
- Virtual environment builds successfully.
- Dependencies install without undocumented manual fixes.
- Configuration template is complete and contains no real secret.
- Health endpoint works.
- Automated tests pass.
- Documentation matches the actual commands.
- Demonstration can be repeated from a clean environment.